LARAVEL

TESTS AVEC PHPUNIT

CHAPITRE 1 — INTRODUCTION AUX TESTS

1.1 Qu'est-ce qu'un test ?

Définition :
Un **test** est une procédure automatisée qui vérifie qu'une partie de votre application fonctionne correctement.

Explication simple :
Imaginez que vous construisez une maison. Avant de livrer la maison au client, vous vérifiez :

- Que l'électricité fonctionne (tester les prises)
- Que la plomberie ne fuit pas (tester les robinets)
- Que les portes s'ouvrent et se ferment (tester les poignées)

En programmation, c'est pareil :

- On vérifie que les pages s'affichent (tester les routes)
- On vérifie que les données s'enregistrent (tester les formulaires)
- On vérifie que les calculs sont justes (tester les méthodes)

1.2 Pourquoi tester est important ?

Sans tests :

```
php
// Vous ajoutez une nouvelle fonctionnalité
public function nouvelleFonction()
{
    // Vous modifiez du code existant
}

// Mais vous ne savez pas si vous avez cassé autre chose !
// Vous devez tout tester manuellement à chaque fois
// → Perte de temps
// → Risque d'oublier des cas
```

// → Stress à chaque déploiement

Avec tests :

```
php
// Vous écrivez les tests une fois
public function test_ancienne_fonction()
{
    // Ce test vérifie que l'ancienne fonction marche toujours
}

// Après chaque modification, vous lancez tous les tests
// Si tout est vert → tout fonctionne
// Si quelque chose est rouge → vous savez exactement quoi corriger
```

1.3 Les différents types de tests

Type de test	Explication simple	Exemple concret
Test unitaire	Teste une toute petite partie (une seule fonction)	Tester que 2+3 fait bien 5
Test HTTP	Teste que les pages web répondent correctement	Tester que la page d'accueil s'affiche
Test base de données	Teste que les données sont bien sauvegardées	Tester qu'un client est bien dans la base
Test de fonctionnalité	Teste un parcours complet	Tester qu'un utilisateur peut se connecter

1.4 PHPUnit dans Laravel

PHPUnit est l'outil de tests utilisé par Laravel.

Où sont les tests ?

```
text
votre-projet/
 ├── tests/              # Dossier des tests
 |   ├── Unit/           # Tests unitaires (petites parties)
```

```
│   │   └── ExampleTest.php   # Test exemple
│   ├── Feature/            # Tests de fonctionnalité (routes, pages)
│   │   └── ExampleTest.php   # Test exemple
│   └── TestCase.php        # Classe de base (ne pas modifier)
```

Comment lancer les tests ?

 # Dans le terminal, à la racine du projet

```
php artisan test

# OU
./vendor/bin/phpunit
```

Résultat attendu :

```
PASS  Tests\Unit\ExampleTest
✓ basic test

PASS  Tests\Feature\ExampleTest
✓ basic test

Tests:  2 passed
Time:   0.5s
```

CHAPITRE 2 — CRÉER ET STRUCTURER UN TEST

2.1 Créer un fichier de test

Commande :

```
php artisan make:test ChambreTest
```

Explication :

- make:test = commande pour créer un test
- ChambreTest = nom du fichier de test

Résultat :

```
Test created successfully.
```

Fichier créé : tests/Feature/ChambreTest.php

2.2 Structure d'une classe de test

```
<?php

namespace Tests\Feature;
// Le dossier où se trouve ce fichier

use Tests\TestCase;
// On importe la classe TestCase de Laravel
// C'est elle qui donne toutes les méthodes de test

class ChambreTest extends TestCase
// Notre classe de test hérite de TestCase
{
    // Les méthodes de test seront écrites ici
}
```

2.3 Structure d'une méthode de test

Une méthode de test a toujours **3 parties** :

```
public function test_exemple()
{
    // 1. ARRANGE (Préparation)
    // On prépare les données nécessaires au test
    $a = 5;
    $b = 3;

    // 2. ACT (Action)
    // On exécute l'action à tester
    $resultat = $a + $b;
```

```
    // 3. ASSERT (Vérification)
    // On vérifie que le résultat est celui attendu
    $this->assertEquals(8, $resultat);
}
```

Explication des 3 étapes :

Étape	Rôle	Exemple réel
Arrange	Préparer le terrain	Créer un client dans la base
Act	Faire l'action à tester	Aller sur la page /clients
Assert	Vérifier le résultat	Vérifier que le client est affiché

2.4 Convention de nommage des tests

```
// On nomme les méthodes pour qu'elles soient compréhensibles
public function test_un_visiteur_peut_voir_la_liste_des_chambres()
public function test_un_visiteur_peut_ajouter_une_chambre()
public function test_le_prix_doit_etre_un_nombre_positif()
public function test_le_numero_de_chambre_doit_etre_unique()
```

Pourquoi ?
Quand un test échoue, on voit tout de suite ce qui ne va pas :

```
FAILED  test_le_prix_doit_etre_un_nombre_positif
// On comprend immédiatement : problème de validation du prix
```

CHAPITRE 3 — LES ASSERTIONS

3.1 Qu'est-ce qu'une assertion ?

Définition :
Une **assertion** est une vérification que le résultat obtenu correspond au résultat attendu.

Explication :
C'est comme un professeur qui corrige une copie :

- Il a la réponse attendue (corrigé)

- Il regarde la réponse de l'élève (résultat)
- Il vérifie si c'est juste ou faux

3.2 Les assertions les plus courantes

assertEquals() – Vérifier l'égalité

```
public function test_addition()
{
   $resultat = 2 + 2;

   // On vérifie que 2+2 est bien égal à 4
   $this->assertEquals(4, $resultat);
   // Si c'est pas 4, le test échoue
}
```

assertTrue() / assertFalse() – Vérifier vrai/faux

```
public function test_utilisateur_est_admin()
{
   $utilisateur = new User(['role' => 'admin']);

   // On vérifie que l'utilisateur est bien admin
   $this->assertTrue($utilisateur->isAdmin());

   // On vérifie qu'il n'est pas banni
   $this->assertFalse($utilisateur->isBanned());
}
```

assertNull() / assertNotNull() – Vérifier null ou pas

```
public function test_chambre_peut_avoir_description()
{
   $chambre = Chambre::find(1);

   // La description peut être null (pas obligatoire)
```

```
    $this->assertNull($chambre->description);

    // Mais le numéro ne doit jamais être null
    $this->assertNotNull($chambre->numero);
}
```

assertCount() – Vérifier le nombre d'éléments

```
public function test_categorie_a_plusieurs_chambres()
{
    $categorie = Categorie::find(1);
    $chambres = $categorie->chambres;

    // On vérifie que la catégorie a bien 3 chambres
    $this->assertCount(3, $chambres);
}
```

assertContains() – Vérifier qu'un élément est dans une liste

```
public function test_liste_contient_chambre_101()
{
    $chambres = ['101', '102', '103'];

    // On vérifie que "101" est bien dans la liste
    $this->assertContains('101', $chambres);
}
```

3.3 Tableau récapitulatif des assertions

Assertion	Vérifie que...	Exemple
assertEquals($attendu, $actuel)	les deux valeurs sont égales	$this->assertEquals(5, $resultat)
assertNotEquals($attendu, $actuel)	les deux valeurs sont différentes	$this->assertNotEquals(0, $resultat)
assertTrue($condition)	la condition est vraie	$this->assertTrue($user->estConnecte())
assertFalse($condition)	la condition est fausse	$this->assertFalse($user->estBanni())
assertNull($variable)	la variable est null	$this->assertNull($user->dateSuppression)
assertNotNull($variable)	la variable n'est pas null	$this->assertNotNull($user->email)
assertCount($nb, $collection)	la collection a $nb éléments	$this->assertCount(3, $chambres)
assertContains($valeur, $liste)	la liste contient la valeur	$this->assertContains('admin', $roles)
assertEmpty($variable)	la variable est vide	$this->assertEmpty($erreurs)
assertNotEmpty($variable)	la variable n'est pas vide	$this->assertNotEmpty($resultats)

CHAPITRE 4 — TESTS HTTP

4.1 Tester des routes GET

Test 1 : Vérifier qu'une page existe (code 200)

```
public function test_la_page_liste_des_chambres_existe()
{
    // ACT : on fait une requête GET sur /chambres
    $response = $this->get('/chambres');
```

```
    // ASSERT : on vérifie que le code HTTP est 200 (OK)
    $response->assertStatus(200);
}
```

Explication ligne par ligne :

Ligne	Explication
$response = $this->get('/chambres');	On simule une requête GET à l'URL /chambres et on stocke la réponse
$response->assertStatus(200);	On vérifie que la réponse a le code HTTP 200 (page trouvée)

Test 2 : Vérifier qu'une page 404 fonctionne

```
public function test_une_chambre_inexistante_renvoie_404()
{
    // ACT : on demande une chambre qui n'existe pas (ID 999)
    $response = $this->get('/chambres/999');

    // ASSERT : on vérifie que c'est bien une erreur 404
    $response->assertStatus(404);
}
```

Test 3 : Vérifier le contenu de la page

```
public function test_la_liste_affiche_les_chambres()
{
    // ARRANGE : on crée des chambres dans la base de test
    Chambre::create([
        'numero' => '101',
        'prix' => 1200,
        'capacite' => 2,
        'categorie_id' => 1
    ]);
```

```
    Chambre::create([
        'numero' => '102',
        'prix' => 1500,
        'capacite' => 3,
        'categorie_id' => 1
    ]);

    // ACT : on va sur la page
    $response = $this->get('/chambres');

    // ASSERT : on vérifie que les numéros sont affichés
    $response->assertSee('101');
    $response->assertSee('102');
    // assertSee = vérifie que le texte est présent dans la page
}
```

4.2 Tester des routes POST (formulaires)

Test 1 : Soumission valide

```
public function test_un_visiteur_peut_ajouter_une_chambre()
{
    // ARRANGE : on prépare les données du formulaire
    $donnees = [
        'numero' => '103',
        'prix' => 2000,
        'capacite' => 4,
        'categorie_id' => 1
    ];

    // ACT : on envoie le formulaire en POST
    $response = $this->post('/chambres', $donnees);
```

```
    // ASSERT : vérifier la redirection (code 302)
    $response->assertStatus(302);
    $response->assertRedirect('/chambres');

    // ASSERT : vérifier que la chambre est dans la base
    $this->assertDatabaseHas('chambres', [
        'numero' => '103',
        'prix' => 2000
    ]);
}
```

Test 2 : Soumission invalide (validation)

```
public function test_on_ne_peut_pas_ajouter_chambre_sans_numero()
{
    // ARRANGE : données sans numéro
    $donnees = [
        'prix' => 2000,
        'capacite' => 4,
        'categorie_id' => 1
    ];

    // ACT : on envoie le formulaire
    $response = $this->post('/chambres', $donnees);

    // ASSERT : vérifier qu'il y a une erreur de validation
    $response->assertSessionHasErrors('numero');
    // assertSessionHasErrors = vérifie que le champ 'numero' a une erreur

    // ASSERT : vérifier que rien n'a été inséré
    $this->assertDatabaseMissing('chambres', [
```

```
        'prix' => 2000
    ]);
}
```

4.3 Assertions HTTP spécifiques

Assertion	Description	Exemple
assertStatus(200)	Vérifie le code HTTP	$response->assertStatus(200)
assertOk()	Vérifie code 200 (équivalent)	$response->assertOk()
assertNotFound()	Vérifie code 404	$response->assertNotFound()
assertRedirect($uri)	Vérifie redirection	$response->assertRedirect('/chambres')
assertSee($texte)	Vérifie que le texte est dans la réponse	$response->assertSee('Bienvenue')
assertDontSee($texte)	Vérifie que le texte n'est pas là	$response->assertDontSee('Erreur')
assertSessionHasErrors($champ)	Vérifie erreur de validation	$response->assertSessionHasErrors('email')
assertSessionHasNoErrors()	Vérifie aucune erreur	$response->assertSessionHasNoErrors()

CHAPITRE 5 — TESTS DE BASE DE DONNÉES

5.1 Problème : les tests modifient la vraie base

Sans précaution :

```
public function test_ajout_chambre()
{
    // Ce test ajoute une vraie chambre dans votre base
    $this->post('/chambres', ['numero' => '101', ...]);


    // Votre base de développement est modifiée !
      // Les tests suivants seront perturbés
```

```
}
```

5.2 Solution : base de données temporaire

Dans le fichier phpunit.xml **:**

```
<php>
   <env name="DB_CONNECTION" value="sqlite"/>
   <env name="DB_DATABASE" value=":memory:"/>
</php>
```

Explication :

- sqlite = type de base de données légère
- :memory: = base en mémoire RAM (pas de fichier)
- À chaque test, la base est créée puis détruite
- Vos vraies données ne sont jamais touchées

5.3 Le trait RefreshDatabase

```
<?php

namespace Tests\Feature;

use Tests\TestCase;
use Illuminate\Foundation\Testing\RefreshDatabase;
// RefreshDatabase = réinitialise la base après chaque test

class ChambreTest extends TestCase
{
   use RefreshDatabase;
   // Grâce à ça, chaque test repart d'une base vide

   public function test_exemple()
   {
      // Ici la base est propre à chaque test
   }
```

```
}
```

Comment ça marche :

1. Avant le premier test → Laravel crée toutes les tables
2. Après chaque test → Laravel vide toutes les tables
3. Chaque test commence avec une base vierge

5.4 Assertions pour la base de données

assertDatabaseHas() – Vérifier qu'un enregistrement existe

```
public function test_chambre_bien_ajoutee()
{
    // ACT : on ajoute une chambre
    $this->post('/chambres', [
        'numero' => '101',
        'prix' => 1200,
        'capacite' => 2,
        'categorie_id' => 1
    ]);

    // ASSERT : vérifier que la chambre est dans la base
    $this->assertDatabaseHas('chambres', [
        'numero' => '101',
        'prix' => 1200
    ]);
    // Cette assertion vérifie qu'il existe une ligne dans la table
    // 'chambres' qui a ces colonnes avec ces valeurs
}
```

assertDatabaseMissing() – Vérifier qu'un enregistrement n'existe pas

```
public function test_chambre_bien_supprimee()
{
    // ARRANGE : on crée une chambre
    $chambre = Chambre::create([
```

```
        'numero' => '101',
        'prix' => 1200,
        'capacite' => 2,
        'categorie_id' => 1
    ]);

    // ACT : on la supprime
    $this->delete('/chambres/' . $chambre->id);

    // ASSERT : vérifier qu'elle n'est plus dans la base
    $this->assertDatabaseMissing('chambres', [
        'id' => $chambre->id
    ]);
}
```

assertDatabaseCount() – Vérifier le nombre d'enregistrements

```
public function test_categorie_a_bien_3_chambres()
{
    // ARRANGE : on crée 3 chambres pour une catégorie
    Chambre::factory()->count(3)->create([
        'categorie_id' => 1
    ]);

    // ASSERT : vérifier qu'il y a bien 3 chambres
    $this->assertDatabaseCount('chambres', 3);
}
```

5.5 Tableau récapitulatif DB

Assertion	Description	Exemple
assertDatabaseHas($table, $conditions)	Vérifie qu'une ligne existe	$this->assertDatabaseHas('users', ['email' => 'test@test.com'])
assertDatabaseMissing($table, $conditions)	Vérifie qu'une ligne n'existe pas	$this->assertDatabaseMissing('users', ['id' => 1])
assertDatabaseCount($table, $nb)	Vérifie le nombre de lignes	$this->assertDatabaseCount('users', 5)
assertSoftDeleted($table, $conditions)	Vérifie soft delete	$this->assertSoftDeleted('users', ['id' => 1])

CHAPITRE 6 — LES FACTORIES POUR LES TESTS

6.1 Qu'est-ce qu'une Factory ?

Définition :
Une **factory** est une usine à données factices pour les tests.

Problème :
Pour tester, on a besoin de données dans la base.
Les créer manuellement est long :

```
// À faire pour chaque test

Chambre::create([

    'numero' => '101',

    'prix' => 1200,

    'capacite' => 2,

    'categorie_id' => 1

]);
```

Solution avec factory :

```
// Une seule ligne

Chambre::factory()->create();

  // Crée automatiquement une chambre avec des données aléatoires
```

6.2 Créer une Factory

Commande :

php artisan make:factory ChambreFactory

Fichier créé :

database/factories/ChambreFactory.php

6.3 Code d'une Factory

```
<?php

namespace Database\Factories;

use App\Models\Chambre;
use Illuminate\Database\Eloquent\Factories\Factory;

class ChambreFactory extends Factory
{
    protected $model = Chambre::class;
    // On indique à quelle modèle cette factory est liée

    public function definition()
    {
        return [
            // fake() = génère des données aléatoires
            'numero' => fake()->unique()->numberBetween(100, 999),
            // Génère un numéro unique entre 100 et 999

            'prix' => fake()->numberBetween(500, 5000),
            // Génère un prix aléatoire entre 500 et 5000

            'capacite' => fake()->numberBetween(1, 4),
            // Génère une capacité aléatoire entre 1 et 4
```

```
        'categorie_id' => 1
        // Pour l'instant on met une valeur fixe
    ];
  }
}
```

6.4 Utiliser les factories dans les tests

Créer un seul enregistrement :

```
public function test_exemple()
{
  // Crée une chambre et la sauvegarde en base
  $chambre = Chambre::factory()->create();

  // On peut maintenant l'utiliser dans les tests
  $this->assertEquals(1, Chambre::count());
}
```

Créer plusieurs enregistrements :

```
public function test_exemple()
{
  // Crée 5 chambres
  $chambres = Chambre::factory()->count(5)->create();

  $this->assertCount(5, $chambres);
  $this->assertDatabaseCount('chambres', 5);
}
```

Créer avec des valeurs spécifiques :

```
public function test_exemple()
{
  // Crée une chambre avec un numéro spécifique
  $chambre = Chambre::factory()->create([
```

```
        'numero' => '555',
        'prix' => 9999
    ]);

    $this->assertEquals('555', $chambre->numero);
    $this->assertEquals(9999, $chambre->prix);
}
```

CHAPITRE 7 — EXEMPLES COMPLETS

Exemple 1 : Tester le CRUD complet des chambres

```
<?php

namespace Tests\Feature;

use Tests\TestCase;
use App\Models\Chambre;
use App\Models\Categorie;
use Illuminate\Foundation\Testing\RefreshDatabase;

class ChambreCrudTest extends TestCase
{
    use RefreshDatabase;
    // Chaque test repart d'une base vierge

    public function test_on_peut_voir_la_liste_des_chambres()
    {
        // ARRANGE : créer 3 chambres
        Chambre::factory()->count(3)->create();

        // ACT : visiter la page
```

```
    $response = $this->get('/chambres');

    // ASSERT : page OK et contient les chambres
    $response->assertStatus(200);
    $response->assertSee(Chambre::first()->numero);
}

public function test_on_peut_ajouter_une_chambre()
{
    // ARRANGE : créer une catégorie pour la clé étrangère
    $categorie = Categorie::factory()->create();

    // Données du formulaire
    $donnees = [
        'numero' => '101',
        'prix' => 1500,
        'capacite' => 2,
        'categorie_id' => $categorie->id
    ];

    // ACT : envoyer le formulaire
    $response = $this->post('/chambres', $donnees);

    // ASSERT : redirection
    $response->assertStatus(302);
    $response->assertRedirect('/chambres');

    // ASSERT : donnée en base
    $this->assertDatabaseHas('chambres', [
```

```
        'numero' => '101',
        'prix' => 1500
    ]);
}

public function test_numero_doit_etre_unique()
{
    // ARRANGE : créer une chambre avec numéro 101
    Chambre::factory()->create(['numero' => '101']);

    $categorie = Categorie::factory()->create();

    $donnees = [
        'numero' => '101', // Même numéro
        'prix' => 2000,
        'capacite' => 3,
        'categorie_id' => $categorie->id
    ];

    // ACT : tentative d'ajout
    $response = $this->post('/chambres', $donnees);

    // ASSERT : erreur de validation
    $response->assertSessionHasErrors('numero');

    // ASSERT : toujours 1 chambre dans la base
    $this->assertDatabaseCount('chambres', 1);
}

public function test_on_peut_modifier_une_chambre()
```

```
{
    // ARRANGE : créer une chambre
    $chambre = Chambre::factory()->create([
        'numero' => '101',
        'prix' => 1000
    ]);

    // Nouvelles données
    $donnees = [
        'numero' => '102',
        'prix' => 2000,
        'capacite' => $chambre->capacite,
        'categorie_id' => $chambre->categorie_id
    ];

    // ACT : modification
    $response = $this->put('/chambres/' . $chambre->id, $donnees);

    // ASSERT : redirection
    $response->assertRedirect('/chambres');

    // ASSERT : données mises à jour
    $this->assertDatabaseHas('chambres', [
        'id' => $chambre->id,
        'numero' => '102',
        'prix' => 2000
    ]);
}

public function test_on_peut_supprimer_une_chambre()
```

```
    {
        // ARRANGE : créer une chambre
        $chambre = Chambre::factory()->create();

        // ACT : suppression
        $response = $this->delete('/chambres/' . $chambre->id);

        // ASSERT : redirection
        $response->assertRedirect('/chambres');

        // ASSERT : chambre supprimée
        $this->assertDatabaseMissing('chambres', [
            'id' => $chambre->id
        ]);
    }
}
```

CHAPITRE 8 — BONNES PRATIQUES

8.1 Ce qu'il faut tester

Élément	À tester ?	Exemple
Routes	Oui	Vérifier que chaque URL répond
Formulaires valides	Oui	Vérifier que l'insertion fonctionne
Formulaires invalides	Oui	Vérifier que la validation bloque
Pages avec données	Oui	Vérifier que les données s'affichent
Méthodes complexes	Oui	Vérifier les calculs, les algorithmes

Élément	À tester ?	Exemple
Code simple (getters)	Non	Perte de temps

8.2 Structure d'un bon test

```
public function test_description_claire()
{
   // 1. ARRANGE : préparer les données
   // 2. ACT : exécuter l'action
   // 3. ASSERT : vérifier le résultat
}
```

8.3 Nommage des tests

```
//  Mauvais nom
public function test_chambre()

//  Bon nom (explique quoi et résultat attendu)
public function test_une_chambre_peut_etre_ajoutee()
public function test_le_numero_doit_etre_unique()
public function test_la_liste_affiche_toutes_les_chambres()
```

8.4 Commandes utiles

```
# Lancer tous les tests
php artisan test

# Lancer un fichier spécifique
php artisan test tests/Feature/ChambreTest.php

# Lancer une méthode spécifique
php artisan test --filter test_ajout_chambre

# Lancer avec plus de détails
```

```
php artisan test --verbose
```